Doc. No.: P1043r0
Date: 2018-05-07
Audience: LEWG, LWG, EWG
Reply to: Andrzej Krzemieński (akrzemi1 (at) gmail (dot) com)
Nevin Liber (nevin (at) cplusplusguy (dot) com)

Narrow contracts in `string_view` versus P0903R1

> *And then I went and invented the null pointer. You either have to check every reference, or you risk disaster.*
>
> — Tony Hoare, "Null References: The Billion Dollar Mistake"

Currently `string_view`'s converting constructor from `const char *` has a precondition (narrow contract): invoking it with a null pointer is undefined behavior. Paper P0903R1 proposes to widen the contract so that passing a null pointer is equivalent to calling the default constructor instead. The declared motivation for the change is to be able to migrate `const char*` interfaces to `std::string_view`. That proposal triggered a long discussion in the reflector. In this paper we provide the summary of the discussion. In particular, we describe the following:

1. What is the purpose of `string_view`.
2. Does/can/should `string_view` hold a not-a-string value, which is distinct from an zero-sized string value.
3. Should `nullptr` passed as `const char *` indicate a not-a-string.
4. Can `string_view` be used as a replacement for `const char *` in interfaces?
5. How do C and C++ treat null pointers that are supposed to represent strings.
6. The scope of the change: should it be only `string_view`'s constructor, or also `string`'s constructor.
7. What is gained by keeping the contract narrow, and what is gained by making it wide.

0. Definitions

In this document we use concept *String*. By this we mean a contiguous sequence of characters; in particular, a sequence may be of size zero. This concept is independent of any representation on a machine; in particular it does not depend on whether the representation requires a pointer to some remote data, or if the size of the string is denoted by a trailing null character. The only thing that the concept is concerned with is the number and the value of the characters that constitute the sequence.

We do not use term "empty", as it turned out to be ambiguous and caused some confusion. Instead, we use two terms to denote two different values: a *zero-sized* string, and *not-a-string*. The not-a-string value is not part of concept String.

1. The purpose of `string_view`

The purpose of `string_view` is indicated in N3442. Previously, a uniform way of accepting a string in any form (`std::string`, a string literal, `std::vector<char>`, array of characters of known size, pointer to a null-terminated array of characters), was to use signature:

```
void f(const std::string& s);
```

This often required the creation of a temporary object of type `std::string` which potentially allocated heap memory (both pre-C++11 and when the object does not fit into the small string optimization) and copied characters. This copying is not really necessary, given that the same sequence already exists somewhere. Now, `std::string_view` offers the ability to represent a string in all the above forms but without incurring any resource allocation or copying of characters.

A second, similar motivation is that `string_view` allows to take its sub-strings cheaply, without the necessity to allocate a second buffer for storing a null-terminated substring.

However, `string_view`'s interface is slightly different; in particular, `sv.data()` is not required to point a null-terminated sequence of characters:

```
std::vector<char> v {'A', 'B', 'C'};
std::string_view sv(v.data(), v.size());
// no null in array under sv.data()
```

This is the goal and the motivation for `std::string_view`. But now that we have it, we can explore if it could fit other purposes as well, for instance as a replacement for interfaces that currently only accept pointers to null-terminated char sequences, or a replacement for interfaces that currently accept `const char *` with semantics slightly different than these of pointers to null-terminated char sequences. The motivation for P0903R1 is one such interface, which allows a null pointer as valid input.

While the extension of the use cases for `std::string_view` are worth exploring, they should not compromise the first and primary goal: uniformly handle the cases previously handled by `const std::string &` (but faster), without compromising the functionality, safety and performance features.

1.1. `string_view`'s contract

The contents of `string_view` consist of a pointer to the beginning of the character sequence and the size of the sequence. This bears similarity to `pair<const char*, size_t>`. However, `string_view` cannot be thought of as `pair<const char*, size_t>`. The latter simply contains a pointer and a number, and assumes no connection between the two pieces of data: they are just two numbers; e.g., `{nullptr, 10}` is fine, or a random address and a random number is fine. This is reflected in `pair`'s `operator==`: two pairs compare equal if their corresponding members all compare equal.

In contrast the intended usage of `string_view` is:

```
for (char ch : sv)
  process(ch);
```

Therefore the type has an *invariant*: the value of `sv.data()` is such that `sv.data()[0]`, `sv.data()[1]`, ..., `sv.data()[sv.size() - 1]` are valid references. Or, in other words, `{sv.data(), sv.size()}` should represent a valid counted range. Also, `operator==`, which defines the value of any type, takes into account only the values of these characters, not the addresses. Thus two `string_view`s can contain different pointers, but still compare equal. This also implies a precondition on the constructor taking a pointer and a size: these two shall represent a valid counted range.

A zero-sized string is represented by a `string_view` object `sv` when `sv.size() == 0`, which includes objects created as follows: `string_view{nullptr, 0}` or `string_view{"A", 0}`. For zero-sized strings the pointer `sv.data()` need not be dereferenced to observe the value of the string. `string_view` objects cannot store a not-a-string value. This is discussed in detail in section 4.

2. Passing null pointer as `const char*`

2.1. The C-string interface

In C and C++ special semantics are associated to single arguments of type `const char *` that are expected to represent values of concept String. We will refer to these semantics in this paper as the *C-string interface*. These semantics are:

- The pointer represents an address to an array of characters.
- While iterating through these characters we are guaranteed to observe a character of value `'\0'` at some point.

- The values of the characters up to but excluding the first observed character `'\0'` constitute the value of the string.

This implies a number of things:

1. The size/length of the string is the distance from the address denoted by the pointer to the address of the `'\0'` character. The size is determined by observing the values of the characters; not from the numeric value of the input pointer.
2. An array represented by literal `""` represents a zero-sized string even though the array's size is 1. Similarly, a pointer obtained from expression `("A" + 1)` represents a zero-sized string.

This also implies a precondition on the input:

- The pointer cannot be null (UB otherwise).
- The pointer must point to a valid array of characters, or a single character with value '\0' (UB otherwise).
- The array pointed to must contain a character with value '\0' (UB otherwise).

Clearly, this constructor has a precondition, and even applying P0903R1 cannot remove it: it can only widen the precondition slightly.

There are other ways to assign different semantics to type inputs of type `const char*` than the one described above. The *C-string interface* will be paid to the most attention in this paper.

2.2. Null pointers as `const char*` in C

Whenever functions in C want a single argument of type `const char*` to represent a string they use the C-string interface. This is reflected in the C Standard:

7.1.1/1

> A *string* is a contiguous sequence of characters terminated by and including the first null character. [...] A *pointer to a string* is a pointer to its initial (lowest addressed) character. The *length of a string* is the number of bytes preceding the null character and the *value of a string* is the sequence of the values of the contained characters, in order.

7.1.4/1

> Each of the following statements applies unless explicitly stated otherwise in the detailed descriptions that follow: If an argument to a function has an invalid value (such as [...] a null pointer[...]) [...], the behavior is undefined.

C functions with C-string interface include `atoi`, `puts`, `fprintf`, `fopen`, or string manipulating functions like `strcpy`, `strcat`.

There are some exceptions to that rule. First, some functions do specify what happens when a null-pointer is passed and there are good reasons to pass one: `setlocale` (null pointer means "only get current locale, don't set anything"), `system` (null -> determine if command processor exists, non-null -> pass command to processor), `mblen` (not-null -> determine the number of bytes in multi-byte character, null -> determine if state-dependent encodings are supported). In all these cases passing null pointer simply triggers a different path in the functions. This is the C way of reducing the number of function names. In C++ we would probably have two functions per each null and non-null input.

Second, functions with bounds-checking interfaces, such as `strcpy_s` do have a defensive check for null-pointer inputs. Such an input is considered an error and results in the call to a *constraint handler* function: it might call `abort()` or ignore the situation by default, or do whatever handler is installed by in the program.

Third, functions like `strncpy` determine the length in a mixed way. An additional length `len` is provided, if `'\0'` cannot be found in the first `len` locations, value `len` is used as string length (and is not `'\0'`-terminated). This departs from the C-string interface, but still treats null-pointer input as invalid (UB).

Fourth, functions like `memcpy` do not take `const char*`, but `const void*` arguments, but they are still relevant in the discussion. In this case the length of the sequences is provided separately by additional argument of type `size_t`. Passing null pointers is still UB.

Fifth, this is a function output rather than input, but function `getenv()` returns the pointer to a found string, but if the string hasn't been found a null pointer is returned to indicate that. The return value is either a string or a not-a-string.

2.3. Null pointers as `const char*` in C++

C++ also follows C-string interface whenever it uses single `const char*` argument to represent a string. This includes constructors of `std::fstream`, `std::string`, `std::filesystem::path`, `std::regex`, or argument to `string::find_first_of()`, `std::char_traits::length()`, or `std::ostream::operator<<`.

Occasionally, two `const char*` pointers are used to represent a string, in the same way as ranges are represented in STL by two iterators `[begin, end)`. This is the case for functions like `std::ctype::is`, `std::ctype::do_is` dealing with locales. These can be null pointers provided that `begin == end`. This is logical: because the length of the string can be determined without dereferencing the pointer, the pointer is allowed to be null.

Some functions in `std::char_traits` take two pointers and the size: `ct::copy(s, p, n)`, `ct::move(s, p, n)`. In each case the string is represented by the pointer to the beginning of the string and the size: a *counted range*. Actually three arguments represent two ranges: `[s, s + n)` and `[p, p + n)`. Again, because we do not need to dereference the pointer to determine the string's length, the pointers are allowed to be null.

Another similar situation:

```
template <class InputIt, class OutputIt>
OutputIt copy(InputIt first, InputIt last, OutputIt d_first);
```

This algorithm can be instantiated with three `const char*` values. These three values represent two ranges: `[first, last)` and `[d_first, d_first + distance(first, last)]`. And again, all three pointers can be null (they would represent two zero-sized ranges), because the sizes of the underlying sequences can be determined without dereferencing the pointers.

A question has been asked, if and how the Ranges TS handles the case of a null pointer passed as type `const char*`. The answer is, Ranges TS describes constraints on inputs representing *ranges* denoted by begin and end. The C-string interface is not a range in the sense of Ranges TS. If we want to talk about the analogy to ranges, passing a null pointer to a function with the C-string interface would be analogous to calling:

```
std::for_each(v.begin(), fun); // no v.end()
```

and expecting that the algorithm will deduce that we intended a zero-sized range.

2.4. Other possible semantics of `const char*` arguments

The above are the semantics associated with type `const char*` in C-string interfaces. They are encouraged by the Standard library. But one can imagine functions in other libraries that interpret the value of type `const char*` differently, especially the null pointer value.

First, you can treat the null pointer value as a zero-sized string (`""`). Whenever you call `f(nullptr)` the effect is the same as if you called `f("")`.

Second, you can treat the null pointer value as distinct from any sting, even zero-sized string. This implies that `const char *` is similar to `optional<string>`: `optstr == nullopt` is a different state than `optstr == ""s`, and calling `f(nullptr)` can give different results than calling `f("")`.

Also, type `const char*` can be used for other purposes than representing a string: it can represent an address of a single character. Migrating such usage to `string_view` would be unwise.

2.5. Is passing a null pointer where a string is expected fundamentally wrong?

Much of controversy around P0903R1 is about whether it is valid to pass a null pointer where `const char*` is expected to represent a string. If by a "string" we mean the String concept: a sequence of 0 or more characters, then clearly there is no need to intentionally use the null pointer value. Any value of String, including zero-sized string, can be represented by null-terminated byte sequence that the pointer needs to point to. Plus, it is a well established idiom in C and C++ Standard Library that passing null pointer in such case is illegal, and therefore is an indication of the programmer bug. When such bug is diagnosed the code is changed so that a non-null pointer is passed to function `f()` or function `f()` is not called at all. So, at least when dealing with the Standard Library functions, passing null pointer that is supposed to indicate a string happens only temporarily, and inadvertently, and is ideally corrected as soon as possible.

But this is only the C-string interface. In custom libraries developers can associate other semantics with a single `const char *` parameters intended to represent strings. They will typically be similar to the C-string interface except that null pointer value is treated in a different manner, which can be one of:

1. Null pointer represents the zero-length string.
2. Null pointer represents a yet another value distinct from any sequence of characters. This is equivalent to `optional<string>` not containing a value, which is different even from a zero-sized string.
3. Null pointer value is unwelcome and indicates a bug, but we want functions to accept it and deal with the bug internally at run-time, through defensive if statements or exceptions or something similar.
4. Null pointer value is unwelcome and indicates a bug, but we expect the function to accept it and not cause language-level UB, but we have no further expectations of what happens.

In a third-party library with lots of functions taking single `const char *` to indicate a string with semantics different than the C-string interface, different functions may have different answers to the above question. This means that one cannot automatically refactor all these functions to take `some_string_view` in place of `const char *`, Instead, for each of these functions the semantics need to be determined and only then an appropriate refactoring action can be taken.

During the discussion we have witnessed people saying, "I have passed null pointer as `const char*` and it was always obvious that it indicates a zero-sized string", other said that in their code base null pointer always indicates a not-a-string. Other replies are that such string is empty (different opinions whether it means zero-sized or not-a-string, or if it matters at all), or is "null" (whatever that means for strings). Whatever answer you chose, you will serve one group, and surprise others, causing bugs.

Whatever answer 1, 2, 3, 4 is chosen, the semantics are different than these for the C-string interface. C-string interface, being selected in the Standard Library of C and C++, is automatically a recommendation to the developers. Its UB cases (preconditions) are designed carefully to bring closer the concrete type `const char *` to the abstract notion of a string. Many libraries also adapt the C-string interface, and expect its preconditions as a safety feature. For instance Boost libraries like Lexical Cast, Regex, String Algo. For the programmers that choose to adapt the C-string interface, weakening its preconditions means that the concrete type `const char *` loses the connection with the concept of a string. This will be discussed later.

3. Can `string_view` be used to replace `const char*` interfaces?

The question in the section title is ambiguous. This reflects the ambiguity in the discussions around P0903R1. An interface is not only a type alone, but also the semantics associated with the type. Type alone, especially a general purpose basic type like `const char*`, can be interpret by different functions in different ways.

Here is one example:

```
bool is_default_separator(const char * ch) { return *ch == '-'; }

const char SEPARATOR = '/';
return is_default_separator(&SEPARATOR);
```

Type `const char *` is assumed to be a reference to a single character, here represented by the object's address. No terminating null character is assumed. Migrating this usage to `string_view` would be an error. There is no concept of string associated with this usage of type `const char *`.

The above is a very obvious case, but consider migrating the C-string interface to `string_view`. We have a user-defined function with C-string interface which forwards to the Standard Library function with C-string interface:

```
void display(const char* message)
{
  std::puts(message);
}
```

If we change the interface of function `display()` to take `string_view` we need to answer the question what should be passed to function `std::puts()`. The only candidate matching by type is `message.data()`:

```
void display2(string_view message)
{
  std::puts(message.data());
}
```

But `message.data()` after refactoring has different semantics than pointer `message` before refactoring. Before refactoring we had guarantee that the pointed to array is null terminated. After refactoring this guarantee does not exist. Well, it exists only for a short time after refactoring because the only usages by the callers were through the C-string interface. But now that the type of the argument has changed, so has the contract: callers are sent a message that they can also pass strings that are not null-terminated:

```
display2({"abcd", 2}); // supposedly allowed
display2({vec.data(), vec.size()}); // supposedly allowed
```

Technically, to fix it one can add a precondition on `display2()`: "`message.data()` should point to a null-terminated sequence", but this is probably nobody's intention.

This illustrates that a top-bottom approach to migrating C-string interfaces to `std::string_view` is impossible in general, unless you know that the argument is not passed down to another C-string interface. Bottom-up conversion is more generally possible and preferable.

This also illustrates that it is not possible to mechanically migrate even C-string interfaces to `std::string_view`. At least not all of them. Regardless if we accept P0903R1 or not.

Also, when changing the interfaces from `const char *` to `std::string_view` one needs to have a clear purpose: why was `const char*` bad? What is better after such change?

One answer to this question is that we do not want to compute and recompute the string length time and again by iterating through the string, especially when the size is known statically: we want to store it directly.

Another answer may be, "we do not want `const char*` because it allows the null pointer value which muddles the design: we want to avoid answering the question what to do on null pointer for every function." In this case, changing `std::string_view` modified by P0903R1 only moves the ambiguity to `std::string_view`. The null pointer from `const char*` can now be smuggled wherever `std::string_view` is used. We have deprived ourselves of the old narrow-contract `std::string_view` that was able to introduce clear semantics (string and nothing else) to the parts of program that got rid of unclear (maybe string, maybe not-a-string) semantics.

4. Holding not-a-string value in `std::string_view`

P0903R1 simply proposes to represent a `string_view` converted from null pointer as `{nullptr, 0}`, without going into details of what use would be made of this value. In the Standard Library component we have to answer a question: should a value in a `string_view` converted from null pointer be distinguishable from a zero-sized string?

4.1. Containers may use `{nullptr, 0}` to represent zero-sized string

One thing that needs to be observed is that value `{nullptr, 0}` can be produced when constructing a `string_view` that point to a zero-sized string. This is because a `string_view` can be constructed from the contents of `vector<char>`, and the default constructed `vector<char>` is allowed to return `vec.data() == nullptr`. And in fact the libstdc++ implementation does this:

```
std::vector<char> vec {};
assert (vec.data() == nullptr); // on some implementations

std::string_view sv {vec.data(), vec.size()};
assert (sv.data() == nullptr); // on some implementations
assert (sv.empty());
```

The specification of `std::vector` works under a correct assumption that null-pointer address can be used to mean "no heap memory allocated". P0903R1 works under another assumption that null-pointer address in `sv.data()` at different level of abstraction could mean "a not-a-string rather than a zero-sized string", but this is incorrect as it is incompatible with the lower-level meaning of a null-pointer address. `std::vector` is legally allowed (and even expected) not to allocate any memory to represent a zero-sized sequence. Thus you can have a zero-sized sequence and a null-pointer address indicating no heap memory.

This means that if P0903R1 is adopted, inside functions taking `string_view` it will be impossible to tell if the argument was constructed from a zero-sized string or from a special not-a-string value. This precludes the usages of `string_view` in places where a distinction needs to be made between not-a-string value and zero-sized-string value.

Worse still, because on different implementations `std::vector` may return a non-null-pointer upon default construction, program written in one implementation (where `vec.data()` returns null pointer) that uses P0903R1-like `string_view`s may be thoroughly unit-tested, but when compiled and run with a different implementation of the Standard Library will expose a different behavior.

One could say that having UB in `std::string_view{static_cast<const char*>(nullptr)}` has the same problem (may pass tests on one implementation, but fail on another). But it does not: the problem is similar but different in one important aspect. In UB-friendly `string_view` when someone (like a tester or an end user) observes a bug, it is consistent with the developer's perception; a developer will say, "yes, this code caused an UB, there is something wrong with it, we need to fix it". Therefore programmers are compelled to track and fix the same things that testers and end users consider a bug. In contrast, in P0903R1-like `string_view`s, when a tester or an end user observes a bug, the programmer at a corresponding place sees a well defined behavior and an if-statement, especially put for this condition, and he says, "no, I am 'handling' this case; there must be something wrong with the user's expectations or a platform or something else". In this attitude, detecting and removing bugs is impeded. This is discussed in more detail in section 6.

Also, one could ask if the same problem does not already occur in the current `std::string_view` when it is default-constructed. The answer is *no*. In the current model, `std::string_view`'s default constructor simply refers to a zero-sized string. No not-a-string value exists. No one should have any need to observe the numeric value of address `sv.data()` alone. This makes `string_view` compatible with `std::string` which also represents a zero-sized string when default-constructed.

4.2. Reliable implementation of not-a-string

Holding a distinct not-a-string value would be possible if another value, different than `{nullptr, 0}` is chosen. For instance:

```
inline const char private_global = '\0';

string_view::string_view(const char* s)
  : _data(s ? s : &private_global)
  , _size(traits::length(_data))
{}
```

But it is not clear if anyone wants this.

4.3. A non-regular interface

Also, even the implementation in section 4.2 has its problems. `operator==`, which traditionally defines the value of an object, compares the contents of the strings: number of and values of the characters. It is not able to distinguish not-a-string value from zero-sized string value. `std::hash<std::string_view>` does not distinguish not-a-string value from zero-sized string value. This will make any function that tries to test for special not-a -string value not regular: functions that attempt to distinguish the special not-a-string state and trigger a different control path will give different results for equal inputs (as defined by `hash`, `operator<`, `operator==`). Putting results of such functions to maps, sets, doing memoization, marshalling-unmarshalling, all these will break.

Of course, this is all acceptable if the requirement is, "anything else than immediate language-level UB". But we will not pursue that path in this document.

This above discussion implies that, unless more extensive changes are made to `string_view`, there is only one self-consistent conceptual model for `string_view` that accepts null pointer as `const char *`.

Just treat it as any other zero-sized string. This is acceptable in programs where people want to have two ways of saying zero-sized string: `f("")` and `f(nullptr)`. This is also acceptable in programs that already treat a zero-sized string as unacceptable value: when all functions have a defensive if up front for the zero-sized string:

```
X* foo(const char* p) // desired: p != nullptr && p is not ""
{
  if (p == nullptr || *p == '\0') {
    log_error();
    return nullptr;
  }

  return process(p);
}

X* bar(const char* p) // nullptr is fine
{
  if (p == nullptr)
    p = "";

  return foo(p);
}
```

In this case conflating not-a-string with a zero-sized string is acceptable.

There is another model, which is self-consistent provided that no attempt at memoization is made. (This includes the attempts of storing the results of functions discriminating `sv.data() == nullptr` in maps.)

The not-a-string state is never expected to occur, but if it occurs it is treated as a bug in the program and functions that obtain it should immediately stop execution, e.g. by calling `std::terminate()`, throwing an exception, or performing a comparable evasive action. In such case departing from regular/value semantics does not matter, as the program or a part thereof will be shut down anyway.

```
X* baz(const char* p) // desired: p != nullptr
{
  if (p == nullptr) {
    SIGNAL_BUG();
    return nullptr;   // may still be needed!!
  }

  return process(p);
}
```

This second model can still break memoization: a function that discriminates `sv.data() == nullptr` might want to signal the bug and terminate the program, but the value of `sv` (measured with `std::hash<>` or `operator<` but not `sv.data() == nullptr`) has been previously memoized, but at that time the state had not been `sv.data() == nullptr`.

5. Interchangeability of `std::string` and `std::string_view`

Given that `std::string_view` is intended to be a faster replacement for `const std::string&` a question need to be answered whether we can change the interface of one without changing the interface of the other.

P0903R1 proposes to widen the contract of `std::string_view`'s converting constructor. According to the theory of *design by contract*, in a correct (i.e., bug-free) program, a function with a narrower precondition `f1()` can be replaced by a function with a wider precondition `f2()` and it does not affect program correctness (measured by function inputs satisfying their preconditions). But you cannot replace `f2()` back with `f1()`, because then the contract gets narrower. The only case when you can change from `f1()` to `f2()` and back is when both functions have identical contract.

This is relevant in practice when dealing with functions with `std::string` as parameters. There are situations where you need to change the signature from `void f(std::string)` to `void f(const std::string&)` for performance reasons; and sometimes you have to change the signature from `void f(const std::string&)` to `void f(std::string)`: also for performance reasons, depending on the situation (in case you need to make a copy of the argument inside the function). Contract-wise both these changes are correct: both signatures have the same contract.

Similarly, now that we have `std::string_view`, one may need to change the signature from `void f(const std::string&)` to `void f(std::string_view)`, and sometimes from `void f(std::string_view)` to `void f(std::string)`. Now, if both `std::string` and `std::string_view` have the same narrow contract in constructors (null pointer disallowed in both), the change in either direction is correct contract-wise. (Technically, `std::string` and `std::string_view` have different set of member functions, so mechanically replacing one type with the other may result in a program that does not compile. But in most of the cases we are only interested in the sequence as a whole and the only members that are used are `.begin()` and `.end()`.) But if `std::string_view` had a wider contract in constructor (null pointer is a valid input), migrating from `void f(std::string_view)` to `void f(std::string)` might introduce bugs if someone has started passing null pointers to function `f()`.

Widening the contract for `std::string_view`'s constructor breaks the interchangeability of `std::string` and `std::string_view` in function arguments.

Also, the theory of *design by contract*, which was described for the purpose of Eiffel programming language, does not take into account specifics of C++, where UB is in fact a guarantee that the programmers (at least some of them) rely on. For instance, specifying UB for conversion from null pointer to `std::string` is a guarantee that I can agree with the vendor of libstdc++ that the said conversion will throw an exception, and that the Standard will not try to compromise my agreement with the vendor. This will be considered in more detail later.

If widening the contract of `std::string`'s converting constructor is considered, one should answer another question. Should the same widening be applied to other types, such as `std::filesystem::path`? Should other member functions of `std::string` be also widened their contract? For instance, `string::find_first_of(nullptr)` is currently UB.

And there is still the issue that `std::string_view::data()` is not guaranteed to return a `'\0'`-terminated string while `std::string::data()` is, so a purely mechanical replacement from `string` to `string_view` may have erroneous run-time behavior.

However, it should be noted that currently `std::string_view` is not fully interchangeable with `std::string`. This is because the latter has a precondition in constructor taking a pointer and a length: the pointer cannot be null even if size is zero. The corresponding constructor in `std::string_view` does not have this preconsition.

6. What is gained by keeping the contract narrow

6.1. Simpler conceptual model

Preconditions bring C++ types closer to the real-life concepts that they describe. Consider a function that takes a range of integer values indicated by the lower and upper bounds:

```
bool indicator::is_in_range(int lo, int hi) const
// precondition: lo <= hi
{
  return lo <= _value && _value <= hi;
}
```

Two values of type `int` are a concept in the C++ type-system. A *range* is a concept from the problem domain. Two types of type `int` can represent a range only to some extent. It is denoted by the precondition. If `lo` passed is greater than `hi` there is no way to think of them as a range. The abstraction breaks. One could say, "but let's make `lo > hi` nonetheless a valid input", but then what do you do with this input? One option:

```
bool indicator::is_in_range(int lo, int hi) const
// no precondition
{
  if (lo > hi) return false;
  return lo <= _value && _value <= hi;
}
```

Whatever you do, you are not dealing with a range now, but with two objects of type `int`. There is no notion of being *in range* for two arbitrary `int` values, and the value in the additional return statement is probably wrong, because there is no good answer. The logic of this function has descended from dealing with abstract ranges to performing operations on C++ type-system objects. Reasoning at this level is harder and more bug prone. Intuition fails, and code reviews are less effective.

One could say, given that we have to accept `lo > hi` let's make it useful. Say, in such case change the check form "in range" to "outside range":

```
bool indicator::is_in_range(int lo, int hi) const
// no precondition
{
  if (lo > hi) return _value <= hi || lo <= _value;
  else         return lo <= _value && _value <= hi;
}
```

This is similar to what C functions like `setenv()` do. Now every combination of values triggers a useful output. But now we have completely departed form the notion of a range. We have conflated two concepts into one interface. We have created an unintuitive abstraction. Thinking about it is hard, and bugs very likely to occur.

Similarly, a null pointer value does not represent a string, not even a zero-sized string. Once it is accepted as valid input to `string_view` something has to be done with it: maybe call it a zero-sized string, maybe call it a distinct value from any string value. In either case the abstraction is weakened. We cannot safely think in terms of strings, character sequences, but one level down: about the numeric value of address `sv.data()`.

6.2. Better bug detection with tools

6.2.1. The difference between bugs and precondition violations

A *Bug* (like a type-o, or misunderstanding of the interface, or reading one variable instead of another) is an inadvertent logic in the code that makes the program do something else than what we intended. We cannot define it formally. But we definitely do not want them in our source code. Bugs make programs misbehave and crash. Bugs may cause injuries and fatalities.

An *precondition violation* is a very formal situation. A function (or expression) has a *precondition*: a constraint on the set of values/states. This condition can be specified quite formally, e.g. "`[b, e)` represent a valid range"; sometimes

the precondition can be expressed as a c++ expression, e.g. `i >= 0`. An *precondition violation* is a situation when a function has a precondition (a constraint on input values) and yet it is passed a value/state that does not meet the constraint. This is so formal that we can communicate unambiguously using these terms. Even tools like compilers and static analyzers can understand the notion of precondition violations. Here, we are not talking about bad things that can happen to the outside world, or human intentions. The notion of *precondition violation* is in a different level of abstraction than the notion of a *bug*. Tools are not concerned with potential injuries, but tools can diagnose precondition violations in functions.

A precondition violation is a symptom of a bug; but it is not a bug on itself. By specifying preconditions and making the contracts narrow, you build a connection between the two notions: a precondition violation can now indicate a bug. This is possible only when there are some input values disallowed by the function. The notion of *undefined behavior* or *invalid input* also come close to the notion of precondition violation.

Now, sometimes people are concerned with precondition violations (and invalid inputs) more than with bugs. They claim that if you get rid of preconditions (invalid inputs), you get rid of precondition violations; and when you get rid of precondition violations, you get rid of the problems. In consequence, they postulate to design function interfaces so that any input is "valid", i.e. have no preconditions.

The first part of that claim is actually true: if you get rid of preconditions, you get rid of potential precondition violations. But with this you are only curing the symptoms and let the disease (the bug) spoil your system: and now that you have one symptom less, it will be more difficult to detect it. You simply loosen the connection between the technical notion of "precondition violation" and human notion of "bug". Consider this example containing a bug, where we inadvertently pass a null pointer, whereas we intended to pass a different value:

```
const char * p = "contents.txt";
const char * q = 0;
f(q); // BUG: intended to call f(p);
```

That is, we pass a different pointer than we intended, but the types accidentally match, which forms a well-formed C++ program. First, suppose that function `f()` has a precondition: the argument cannot be a null pointer. If you plant this bug you are violating a precondition. Automatic tools, like static analyzers, do not know our intentions, they cannot recognize such things as "you wanted to pass different argument" or "you confused this name with that one", or "you wanted to pass that value instead", "you have a type-o". But they are good at detecting precondition violations, or illegal values. Invalid input is not a bug itself, but an indication of a bug (in our case: confusing pointers), but if this is reported it is enough for the programmer to kick in and correct it. Static analyzer has *helped* find the bug, but didn't actually find it: the programmer found it based on the information from static analyzer.

Now, consider what happens when you widen the contract. The bug is still there, the pointers are still confused, but there is no precondition violation anymore. Static analyzer is blind and cannot help you detect the bug.

To summarize. The goal is to detect bugs (early, statically). The notion of "precondition violation" or "invalid input" is only a tool that helps detect bugs (or, assert program correctness). The goal is not to detect invalid inputs: it is only a means to the real goal. By widening contracts you render the notion of invalid input unhelpful (or less helpful) in achieving the goal of detecting bugs.

Sometimes people worry more about hitting language-level UB than about bugs in their code. But bugs actually have the same dangerous characteristic as UB: if you hit them, the results are unpredictable. A bug can be thought of as UB at business logic level.

6.2.2. Null pointer as a symptom of a bug

Many commercial programs and online services will give you a "null pointer dereference" error message and fail to process your task. This problem is quite popular even nowadays, not limited to C++. The ability to detect this problem and remove it in time is important. In such cases dereferencing a null pointer is not the source of the problem, it is just a symptom of one. The source of the problem is that we are passing a null pointer value where we are not supposed to. This happens due to omission of the programmer, for instance:

```
void Display::output(std::string_view s);

class Controller
{
  const char * _name = nullptr;

public:
  void display() { Display::output(_name); }
  void setName(const char * n) { _name = n; }
}
```

Function `display()` was called before function `setNumber()` was able to overwrite the null pointer value. In the current version of `std::string_view` this causes a null pointer dereference, and null pointer dereference can be easily detected by various tools today. But this can only be detected if passing null pointer value to `std::string_view`s constructor is UB.

6.2.3. Improved static analysis

Some tools, like static analyzers, in order to detect bugs, need help from the programmer and the libraries: this help is provided via UB.

Any portion of code is suspicious and could technically contain a bug. In order for static analyzers not to produce too many false positive warnings, they need to be certain that a given piece of C++-conformant code does not reflect programmer's intentions. How does the static analyzer know what the programmer's intentions are? It doesn't in general, but sometimes the answer is clear: it is *never* the programmers intention to hit an UB. So, when a static analyzer can find an execution path in the program that triggers an UB, it can with certainty report a bug. But for this to work one needs a *potential* to hit an UB. Making a non-null pointer a precondition is such potential to hit an UB: a good one, because dereferencing a null pointer is well explored and tool-friendly type of UB. As explained above, this does not introduce a potential for bugs: it only makes bugs more visible.

UB is close in nature to a checksum: checksum is redundant, increases the size of the message, and creates the potential for producing invalid messages (where the payload doesn't match the checksum), so someone might say, "remove it, and there will be no way to send an invalid message". But we still want to use checksums, and we all know why.

6.2.4. Flexibility in handling the bug at run-time

When the standard says that the behavior of some operation is undefined, especially in the case of UB as easily diagnosable as dereferencing a null pointer or precondition violation, vendors are allowed to define their guaranteed behavior. Vendors and programmers may use this opportunity to settle on an error reporting scheme for invalid inputs that is best suited for a given program. Ideally, bugs should be removed from the code rather than being responded to at run-time. There is no good universal way to respond to them. A solution good for one project would be detrimental in another. Therefore the best the Standard can do is to leave the decision to engineers that assemble the program from the components, that know the environment that the program will be executed in, and that are best equipped to make the right decision. Such decision might be:

1. Compile the program with UB sanitizer. If null pointer dereference is encountered a message will be logged and the program halted.
2. When the function is evaluated in the `constexpr` context, a compiler error is reported.
3. The vendor's implementation of the library component may throw an exception upon invalid (null pointer) argument to the constructor. In fact, libstdc++ implementation of `std::string` throws an exception when you construct it from null pointer.
4. The vendor's implementation of the library component may use a replacement value which is valid when an original value is invalid. This is what the libstdc++ implementation of `std::string_view` does when you pass null pointer to the constructor: it initializes to `{nullptr, 0}`.
5. You can switch between any of the above based on compiler switches and macro definitions. Vendors can offer different modes in which their implementation of the Standard library operates. In particular for unit-test builds

you can configure the `std::string_view`'s constructor to report unit-test as failed, and then proceed with the zero-sized string value.

These all options are possible only because the Standard leaves undefined what happens. They may not sound like an option if one thinks, "the Standard is my ally, the compiler vendor is my enemy", but if you trust your platform and tools the picture is completely different. In particular, this flexibility means that if Abseil Authors want to implement "go with default-constructed `string_view` upon null pointer" in their implementation of `std::string_view`, that is also a standard-conformant implementation.

But all these options will suddenly be gone if the Standard suddenly defines the behavior to only one right solution. Such one solution cannot serve the entire community.

This illustrates that UB in well designed places is a feature offered to the programmers. Not an omission. Not something to be defined in the future releases of the Standard.

6.3. No need for defensive checks or preconditions

This subsection assumes that the intention of P0903R1 is to modify `std::string_view` so that it can unambiguously store both not-a-string value and a zero-sized string value, and that the callees can later retrieve the information which of the two values was intended. In section 4 we already showed that this is not doable with state `{nullptr, 0}` as proposed by P0903R1.

If `std::string_view` is altered to additionally store the not-a-string value, then in each function taking `std::string_view` needs to make a conscious decision, what the function should do if it receives not-a-string value. Programmers often forget about it, which causes bugs. If they do not forget and they do not want the not-a-string value even to be passed to the function (because in a given context it makes no sense), they have two options:

1. Put a defensive if-statement in the beginning (and there is no good choice as to what to do inside).
2. Put an explicit precondition in the function.

In either case the logic becomes more complicated. In the first case we have an additional branch which is useless in correct programs (that do not pass unintended values). In the second case we have a precondition that anyone needs to be aware of. A precondition is superior to defensive if-statement, but is inferior to strong types that encode the same condition in their *invariants*. Currently in `std::string_view` without P0903R1 we have a *strong invariant*: object of type `std::string_view`, if constructed correctly, and lifetime issues observed, *always* represents a reference to a string. There is no question of "what to do with a not-a-string". Even its default constructor creates a reference to a globally accessible zero-sized string. Everyone who uses type `std::string_view` knows that it deals with strings of different sizes and nothing else. A strong invariant is similar in nature to RAII: if an object managing a resource is in its lifetime, you get the guarantee that the resource is available to you and you do not have to check for anything.

6.4. Performance

In the extension proposed in P0903R1 the constructor taking `const char *` has to perform a test whether the value of the pointer is null before trying to dereference it. When the compiler cannot track how the value of the pointer is obtained, it has to actually perform this check in run-time. In contrast, when the constructor in question adheres to the C-string interface, this check can be skipped. (Admittedly, this is a non-memory-dereferencing `O(1)` check in an `O(n)` interface and the performance difference is more theoretical than practical.)

6.5. Consistency with std::string

While some consider `std::string_view` to have a muddled interface both because a default constructed `std::string_view` is required that `data() == nullptr` as well as having reference semantics instead of value semantics (comparison operators do a deep comparison while copying is shallow), `std::string` is not muddled. While there was not consensus the first time around to change `std::string` to match this change to

`std::string_view`, the question may come up again. Once an interface is muddled, the tendency is to try and muddle it more (or we would not be having this conversation about `std::string_view` in the first place).

7. What is gained by widening the contract

7.1. Handle cases where not-a-string is conflated with zero-sized string

The application of P0903R1 could be helpful in situations where the callers produce null pointer values that indicate not-a-string, which have different semantics than zero-sized strings, and the callees conflate the two values. The following is a motivating example.

```
// caller:
can_compress_ = CheckCompressionType(
  input_headers().GetHeader("User-agent"),       // can be null
  input_headers().GetHeader("Accept-encoding"),  // can be null
  output_headers().GetHeader("Content-type"));   // can be null

// callee:
bool CheckCompressionType(const char *agent,
                          const char *encoding,
                          const char *content_type) {
  if (use_permissive_policy) {
    return SafeToCompressForBlacklist(user_agent, accept_encoding,
                                      content_type);
  }
  return SafeToCompressForWhitelist(user_agent, accept_encoding,
                                    content_type);
}

bool SafeToCompressForWhitelist(const char *user_agent,
                                const char *accept_encoding,
                                const char *content_type) const {
  // If they don't let us know the 3 things we need, we bail
  if (!user_agent || !accept_encoding || !content_type) return false;

  if (!strprefix(accept_encoding, "gzip") &&     // starts with "gzip"
      !strstr(accept_encoding, " gzip") &&       // gzip is a word
      !strstr(accept_encoding, ",gzip")) {
    return false;
  }

  // more logic
  return true;
}
```

Functions `CheckCompressionType()` and `SafeToCompressForWhitelist()` both accept null pointer values; the meaning is, "this piece of data did not come in the request". Unlike a zero-sized string, which means this data came in request and was a zero-sized string.

The goal is to change the interface of function `SafeToCompressForWhitelist()` to take values of type `string_view`. This is because it is easier to describe the logic of the function (inspecting substrings) using the interface of `string_view`. Also, if wee needed to produce a substring at some point, it would be a costly operation on `const char*`. Inside function `SafeToCompressForWhitelist()` not-a-sting value and zero-sized string value are treated uniformly, so conflating both values is acceptable.

If `string_view` is modified as per P0903R1, the refactoring of function `SafeToCompressForWhitelist` becomes simple:

```
bool SafeToCompressForWhitelist(string_view user_agent,
                                string_view accept_encoding,
                                string_view content_type) const {
  // If they don't let us know the 3 things we need, we bail
```

```
  if (user_agent.empty() || accept_encoding.empty() || content_type.empty()) return false;

  if (!accept_encoding.starts_with("gzip") &&
      accept_encoding.find(" gzip") == string_view::npos &&
      accept_encoding.find(",gzip") == string_view::npos) {
    return false;
  }

  // more logic
  return true;
}
```

And nothing needs to be changed in the callers. And there can be hundreds of callers.

A special case of this use case is in programs or their modules where a zero-sized string is already being treated as an invalid value. In such cases when we conflate the invalid null-pointer input with an invalid zero-sized string input, this will not affect the semantics as they will be processed uniformly with defensive checks of the form `sv.empty()`.

7.2. Inability to enforce narrow contracts

Sometimes it is acknowledged that having narrow contracts is desirable, and helps in asserting program correctness. But for economical reasons it is infeasible to migrate a large code-base of a program which previously did not make a distinction into narrow and wide contracts, given the resources that the development team has to spare. In those cases leaving the wide contracts as they are and manually trying address the unexpected input in each function may be the only option within economical reach. In this case one expects every new type to have a wide contract.

8. Alternatives

In this chapter we explore how the use cases of the proponents of P0903R1 can be addressed without changing `std::string_view`, and how the use cases of the opponents of P0903R1 can be addressed once `std::string_view` is changed.

8.1. What can be offered to programmers that want to pass null pointers to `string_view`?

First, make sure if you really need to migrate the argument of type `const char *` to type `std::string_view`. `std::string_view` represents a reference to a string, `const char *` does not necessarily represent a string. Also it might represent the string but with different semantics than you think.

Second, if the goal of the function input is to represent either a string or not-a-string value, use the Standard Library type that is designed to represent this notion of not-a-value: `std::optional<std::string_view>>`. You can use it in interfaces where not-a-string is allowed, and you can still use `std::sting_view` where not-a-string is incorrect. The additional benefit is that your interfaces will clearly indicate for which functions it is correct to pass not-a-string and for which it is an error. But this will require a change in the callee.

Third, if the size of `std::optional<std::string_view>>` or its set of constructors does not match your use case, define your own type that implements semantics in P0903R1:

```
class conflating_string_view : std::string_view
{
  using super = std::string_view;
  constexpr super as_super() const noexcept { return static_cast<super>(*this); }

public:
  using super::super;
  constexpr conflating_string_view (const char * p) noexcept
    : super(p, p ? std::char_traits<char>::length(p) : 0) {}

  // modify the interface of functions taking string_view:
  constexpr size_type find_first_of(conflating_string_view sv, size_type pos = 0) const noexcept {
    return super::find_first_of(sv.as_super(), pos);
```

```
  }

  // just redeclare other members as public:
  using super::empty;
};
```

Different semantics require a different type. And you are offering a different semantics. True, programmers will have to learn a new type. But the alternative is, they will use only one type and will not be aware that at different places the same type has different semantics. And they will only learn it when a bug is found. Similarly here, we encourage to use this type only when you want to provide semantics, "I accept not-a-string value, I conflate it with zero-sized string value". In other cases go with `std::string_view` which has stronger contract.

(Note: it has been previously suggested that such implementation can be written in 5 lines, but these suggestions did not take into consideration that member functions like `find_first_of()` also need the change in contract.)

Fourth, choose the Standard Library implementation that already implements the semantics of P0903R1. The Standard Specifies the behavior as UB, so this is legal for Standard-conforming implementation to implement your desired semantics. If this is not already the case influence your library vendor to implement the behavior of P0903R1, or give you the option to configure the library to do what you want.

Fifth, change the callers, so that they decide how they want to treat the not-a-string value. For instance define function `null_as_empty()` which takes a `const char*` argument interpreted as "either a string or not-a-string" and returns a `const char*` value interpreted as String:

```
constexpr const char * null_as_empty(const char * s) noexcept {
  return s ? s : "";
}
```

Or you can have `null_as_empty` return a dedicated type which can convert to either `const char*` interpreted as string or to `conflating_string_view`. When applying this solution the caller from the example in 8.1 has to be changed to:

```
// caller:
can_compress_ = CheckCompressionType(
  null_as_empty(input_headers().GetHeader("User-agent")),
  null_as_empty(input_headers().GetHeader("Accept-encoding")),
  null_as_empty(output_headers().GetHeader("Content-type")));
```

This requires the change in the caller (which can be considered a disadvantage in its own right), but at the same time it makes the intentions and the logic of the program more clear and penetrable: "from this point we treat not-a-string value and the zero-sized string value in the same way".

Sixth, if the goal is to enable a more convenient/idiomatic implementation of the body of the function, like in section 7.1, this can be addressed by conversion from `const char *` to `string_view` not in the interface but inside the implementation:

```
// this handles strings:
bool private_SafeToCompressForWhitelist(string_view user_agent,
                                        string_view accept_encoding,
                                        string_view content_type) const {
  if (!accept_encoding.starts_with("gzip") &&
      accept_encoding.find(" gzip") == string_view::npos &&
      accept_encoding.find(",gzip") == string_view::npos) {
    return false;
  }

  // more logic
  return true;
}

// this handles the null cases:
bool SafeToCompressForWhitelist(const char *user_agent,
```

```
                                const char *accept_encoding,
                                const char *content_type) const {
  // If they don't let us know the 3 things we need, we bail
  if (!user_agent || !accept_encoding || !content_type) return false;

  return private_SafeToCompressForWhitelist(string_view{user_agent},
                                            string_view{accept_encoding},
                                            string_view{content_type};
}
```

Finally, even if it is not economically possible to prepare the large source code base for migration to narrow-contract components, as described in section 7.2, it may still be possible to identify some smaller components within the source code base, where declaring and enforcing narrow contracts is possible. In those cases the recommendation would be to adapt the narrow-contract `std::string_view` in these components, so at least in these components the correctness of the code can be better asserted.

8.2. What can be offered to programmers that want null pointers passed to `string_view` to remain UB?

Write your own type that handles strings your way, and do not use `std::string_view` *anywhere* in your code. Or derive from the altered `std::string_view` and implement UB in the constructor yourself:

```
class ub_string_view : std::string_view
{
  using super = std::string_view;
  constexpr super as_super() const noexcept { return static_cast<super>(*this); }

public:
  using super::super;
  constexpr ub_string_view(const char * s) noexcept
    : super{s, std::char_traits<char>::length(s)} {} // assuming that length() has narrow contract

  // modify the interface of functions taking string_view:
  constexpr size_type find_first_of(ub_string_view sv, size_type pos = 0) const noexcept {
    return super::find_first_of(sv.as_super(), pos);
  }

  // just redeclare other members as public:
  using super::empty;
};
```

Unlike in 8.1. You need to use `ub_string_view` *everywhere* and (the modified) `std::string_view` is rendered useless (because for optional strings you use `optional<ub_string_view>` anyway).

9. Acknowledgements

Ashley Hedberg and Jorg Brown devoted their time to explain in detail the use cases for null-aware `string_view`, which helped improve the discussion in this paper.

Tomasz Kamiński, Titus Winters and Jorg Brown have reviewed the paper and offered number of useful suggestions that improved the quality of the paper.

10. References

1. Ashley Hedberg, "Define `basic_string_view(nullptr)`".
2. Jeffrey Yasskin, "string_ref: a non-owning reference to a string".
3. Lawrence Crowl, "The Use and Implementation of Contracts".
4. Alisdair Meredith, John Lakos, "`noexcept` Prevents Library Validation".
5. John Lakos, "Normative Language to Describe Value Copy Semantics".

6. Andrzej Krzemieński, "Value constraints".
7. G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, " A Contract Design ".
8. Tony Hoare, "Null References: The Billion Dollar Mistake".